Décisions techniques

Chaque décision est consignée avec son contexte, les alternatives écartées et ce qu'elle coûte. Le but est double : ne pas rediscuter dix fois la même chose, et pouvoir répondre en soutenance à « pourquoi ce choix ? » autrement que par « c'est ce qu'on connaissait ».

---

001 — Back-end en Python / FastAPI

Date : 12 octobre 2026 · **État** : accepté

Contexte. La première version du back était en Node/TypeScript, héritée de SkillMatch. L'énoncé autorise explicitement Node comme Django ou Flask : les deux sont conformes.

Décision. Migrer vers Python 3.12+ / FastAPI / Pydantic / SQLAlchemy.

Alternatives.

- *Rester sur Node* — zéro coût immédiat, mais extraction PDF faible et écosystème moins mûr pour le traitement de textes longs.
- *Next.js en monolithe* — le chemin le plus court vers un déploiement, écarté parce que l'énoncé insiste sur « front-end + back-end » et qu'un monolithe se défend moins bien en deux tiers à l'oral.

Raisons. La documentation OpenAPI est produite automatiquement et sert de pièce au rapport. PyMuPDF respecte l'ordre de lecture sur les mises en page à deux colonnes, fréquentes dans un support de cours. Et tous les ateliers du module sont en Python : le correcteur lit du Python toute l'année.

Conséquences. Positif : `/docs` gratuit, extraction PDF correcte, tests rapides. Coût : deux écosystèmes au lieu d'un, et les types du front doivent rester cohérents avec les schémas du back à la main. Égalité réelle sur la validation — Pydantic et Zod se valent, prétendre le contraire serait malhonnête.

---

002 — Découper les cours longs plutôt que les tronquer

Date : 12 octobre 2026 · **État** : accepté

Contexte. La version précédente coupait à 20 000 caractères et envoyait le début. Un cours de soixante pages était donc résumé à partir de son premier tiers, **sans aucun signalement**. Le pire genre de défaut : le résultat a l'air correct.

Décision. Découper en segments respectant les paragraphes, puis les phrases, avec recouvrement. Résumé par agrégation, quiz sur segments échelonnés, tuteur sur segments sélectionnés.

Alternatives.

- *Augmenter simplement la limite* — les modèles actuels encaissent de très longs contextes, mais la qualité se dégrade au milieu, et le coût croît.
- *Laisser tronquer en l'annonçant* — honnête mais inutile.

Conséquences. Positif : plus rien ne disparaît en silence ; si une limite subsiste, elle apparaît dans les `warnings`. Coût : **le nombre d'appels est multiplié**. Atténué par le cache des résumés, le plafonnement par module, et le choix de la taille de segment — voir décision 012.

---

003 — Sessions synchrones plutôt qu'asynchrones

Date : 12 octobre 2026 · **État** : accepté

Contexte. FastAPI invite à écrire des routes `async`. Le pooler de transactions de Supabase interdit les requêtes préparées, ce qui demande une configuration délicate en asyncpg et produit des pannes difficiles à diagnostiquer.

Décision. Routes `def`, sessions SQLAlchemy synchrones. FastAPI les exécute dans un pool de fils d'exécution.

Raison. Le gain serait nul à notre échelle : quelques étudiants, et un appel au modèle de plusieurs secondes qui domine de toute façon la latence.

Conséquence. À très forte charge il faudrait reconsidérer. Ce n'est pas le cas d'usage.

---

004 — Appeler les fournisseurs en HTTP direct, sans SDK

Date : 12 octobre 2026 · **État** : accepté

Décision. `httpx` plutôt que `google-generativeai` et `anthropic`.

Raisons. La requête est visible dans le code, donc explicable en soutenance. La traduction des défaillances dans notre taxonomie est entièrement maîtrisée — un SDK masque les codes HTTP derrière ses propres exceptions. Et une dépendance de moins par fournisseur.

Conséquence. Il faut suivre soi-même les évolutions d'API. Acceptable : deux fournisseurs, deux points d'entrée.

---

005 — Recherche lexicale (BM25) plutôt que plongements

Date : 12 octobre 2026 · **État** : accepté, à réexaminer

Contexte. Le tuteur doit choisir quels passages envoyer. Deux approches : comparer les **mots** (BM25) ou comparer le **sens** (plongements).

Décision. BM25 pour l'instant.

Alternatives.

- *Plongements par l'API Gemini* — même clé, aucune dépendance nouvelle, déploiement inchangé. Consomme du quota.
- *Plongements locaux via Hugging Face* — ne consomme aucun quota, mais ajoute PyTorch et un modèle de plusieurs centaines de méga-octets, ce qui rend l'hébergement gratuit improbable. Or l'énoncé demande une application « idéalement déployée ».

Raison. Sur un corpus de la taille d'un cours — vingt à soixante pages — la recherche sémantique rapporte peu. Trois segments couvrent déjà une grande partie du document, et le modèle lit très bien ce qu'on lui donne. La sélection n'a pas besoin d'être brillante pour que la réponse soit bonne. Les plongements amélioreraient un maillon qui n'est pas celui qui limite la qualité.

Limite assumée. Une question qui n'emploie aucun mot du cours sera mal servie. Documenté dans `services/segmentation.py`.

À réexaminer. La comparaison BM25 / plongements, mesurée sur un jeu de questions dont on connaît la bonne réponse, est un excellent sujet pour le banc d'essai (AA5). Un résultat négatif mesuré vaut mieux qu'une fonctionnalité ajoutée sans preuve.

---

006 — Anonymiser une fois, avant le découpage

Date : 12 octobre 2026 · **État** : accepté

Contexte. Avec le découpage, le masquage pouvait se faire segment par segment.

Décision. Masquer le cours **entier** d'abord, découper ensuite.

Raison. Des jetons numérotés indépendamment par segment donneraient `[NOM_1]` désignant une personne dans un segment et une autre dans le suivant. L'agrégation mélangerait les identités.

Conséquence. La question de l'étudiant et l'historique doivent être masqués dans le **même** passage que le cours, pour partager la table de jetons. D'où l'astuce de la cloison : les trois textes sont assemblés avec un séparateur unique, masqués ensemble, puis redécoupés.

---

007 — Un seul type d'utilisateur

Date : 10 octobre 2026 · **État** : accepté

Décision. Pas de rôle enseignant, pas d'administrateur.

Raison. Le sujet ne décrit qu'un étudiant qui importe ses cours. Ajouter des rôles serait inventer une exigence.

Conséquence. Pas de garde par rôle : la seule règle d'accès est la propriété, vérifiée côté serveur. Le code en est simplifié, et la sécurité plus facile à démontrer.

---

008 — Garder le masquage des noms malgré sa faible utilité

Date : 12 octobre 2026 · **État** : accepté

Contexte. Le module d'anonymisation vient de SkillMatch, où l'entrée était un CV. Dans un support de cours, le risque est tout autre.

Décision. Conserver le module entier, mais **requalifier sa justification** dans le rapport : les secrets et les coordonnées de tiers, pas les données de l'étudiant.

Raison. L'énoncé l'exige explicitement. Et le risque réel est démontrable : un fichier distribué dans ce module contient un jeton Hugging Face en clair.

Limite écrite. Les patronymes ne sont pas détectés, et un test le documente. Mieux vaut l'énoncer que le laisser découvrir.

---

009 — Base SQLite acceptée en plus de PostgreSQL

Date : 12 octobre 2026 · **État** : accepté

Décision. Déclarer les colonnes JSON en `JSONB` avec une variante `JSON`, et ne passer les options psycopg qu'en PostgreSQL.

Raison. Permet d'exécuter la suite de tests de bout en bout sans base distante ni conteneur. Une chaîne de traitement qu'on ne peut pas exécuter en test n'est vérifiée par personne.

Effet de bord utile. L'application entière tourne en local sans Supabase, ce qui a débloqué la démonstration pendant que le projet Supabase n'existait pas encore.

---

010 — Ports distincts de SkillMatch

Date : 12 octobre 2026 · **État** : accepté

StudyAI : front **5174**, back **3002**. SkillMatch garde 5173 et 3001. Les deux projets doivent pouvoir tourner en même temps.

---

011 — Mélanger les options du quiz côté serveur

Date : 13 octobre 2026 · **État** : accepté

Contexte. À la première série de quiz enregistrés en base, **cinq bonnes réponses sur six se trouvaient en position A**. Un QCM dont la réponse est presque toujours la première n'évalue rien : l'étudiant apprend la position, pas le cours. Le défaut n'était visible qu'en regardant les données, pas en lisant une question isolée.

Décision. Redistribuer la position de la bonne réponse dans le code, après validation et avant enregistrement (`pipeline.melanger_options`).

Alternative écartée. Ajouter au prompt « varie la position de la bonne réponse ». Ce serait confier une garantie à l'obéissance du modèle, alors qu'une ligne de code la rend certaine. Même raisonnement que pour la règle d'anonymisation : ce qui doit être garanti est porté par la structure, pas par la consigne.

Conséquence. Un test unitaire vérifie que les positions varient **et** que le mélange ne change jamais *quelle* réponse est correcte. Un test existant, qui supposait une position fixe, a dû être réécrit : il testait la doublure du modèle, pas la correction.

---

012 — Segments de 24 000 caractères plutôt que 8 000

Date : 13 octobre 2026 · **État** : accepté

Contexte. La taille des segments gouverne directement la consommation de quota : un résumé coûte un appel par segment, plus un appel d'agrégation. Or Google ne publie plus les limites du palier gratuit de Gemini, et plusieurs sources rapportent des baisses importantes courant 2026. Tomber à court de requêtes pendant la démonstration est le risque le plus concret du projet.

Décision. Porter `SEGMENT_CHARS` de 8 000 à 24 000.

Mesure sur le même cours (`Prompt Engineering.pdf` , 26 040 caractères) :

SEGMENT_CHARS	Segments	Appels	Jetons	Latence	Points clés
8 000	4	5	9 389 + 2 299	22 392 ms	10
24 000	2	3	7 391 + 1 398	18 735 ms	8

Raison. 40 % d'appels en moins pour un résumé un peu moins granulaire. La valeur initiale de 8 000 était prudente : elle datait d'avant toute mesure, et les modèles actuels encaissent des contextes bien plus larges.

Ce que ça coûte. Un grain de synthèse plus grossier — huit points clés au lieu de dix. Sur un cours de vingt-six mille caractères, l'écart est réel mais sans conséquence pédagogique.

Ce que ça ne change pas. Aucun contenu n'est perdu dans les deux cas : le cours est traité en entier, et une éventuelle amputation reste signalée dans les `warnings` .

Réversible. C'est une variable d'environnement. Un groupe disposant d'un quota confortable peut revenir à 8 000 sans toucher au code.

---

013 — Des erreurs qui disent quoi faire, et un garde-fou contre la page blanche

Date : 13 octobre 2026 · **État** : accepté

Contexte. Le bandeau d'erreur disait ce qui s'était passé — *« Le quota d'analyses est atteint »* — et s'arrêtait là. L'étudiant restait devant un rectangle rouge sans issue. Par ailleurs, une exception JavaScript dans un composant démontait tout l'arbre React et laissait une **page blanche** : ni message, ni recours, et rien pour comprendre.

Décision. Trois mesures.

1. `lib/erreurs.ts` associe à chaque code une **famille**, une **conduite à tenir** et un indicateur `reessayable`. Le bandeau affiche une icône, le message, le conseil, les actions possibles et le code technique.
2. Un **garde-fou React** (`components/GardeFou.tsx`) entoure les routes — mais pas l'en-tête, pour que l'utilisateur garde sa navigation même si une page s'effondre.
3. Une **page Figma dédiée** rend les onze codes tels qu'ils s'affichent.

Le point qui compte. `reessayable` reflète exactement la politique de reprise du serveur : pas de bouton « Réessayer » sur un quota épuisé, une clef refusée ou un délai dépassé, parce que **le serveur lui-même ne réessaie pas** dans ces cas. L'interface ne promet rien que le back ne tienne.

Alternative écartée. Une page pleine par code d'erreur. Ce serait un recul : quand un résumé échoue, l'étudiant doit garder son cours à l'écran et pouvoir relancer. Le bandeau contextuel est le bon motif pour une erreur récupérable ; les pages pleines sont réservées aux cas sans contexte à préserver — page introuvable, session expirée, erreur inattendue.

Défaut trouvé au passage. Une panne réseau faisait rejeter `fetch` avec un `TypeError`, qui n'a ni code ni message utilisable : l'interface sortait de son propre contrat d'erreur dès que le serveur était injoignable, et affichait « Failed to fetch ». Rattrapé dans `lib/api.ts`, avec un conseil spécifique.

---

014 — reportlab pour la fiche de révision, pas la chaîne Markdown → Chrome

Date : 13 octobre 2026 · **État** : accepté

Contexte. Le projet produit déjà des PDF avec `tools/md2pdf.py`, qui passe par Chrome en mode sans interface. La fiche de révision aurait pu réutiliser cette chaîne.

Décision. Produire la fiche avec `reportlab`, directement en Python.

Raison. `md2pdf.py` convient à une conversion lancée à la main, pas à une requête HTTP : il dépend de la présence de Chrome sur la machine hôte et met plusieurs secondes. reportlab rend la main en quelques dizaines de millisecondes et n'a aucune dépendance système.

Alternative écartée : `weasyprint`, qui convertirait du HTML et permettrait de réutiliser la charte. Il réclame GTK et Pango, pénibles à installer sous Windows — et l'équipe travaille sous Windows.

Conséquence. Une dépendance de plus, et une mise en page décrite en Python plutôt qu'en CSS. Les deux chaînes coexistent : Chrome pour les documents du dépôt, reportlab pour ce que l'application sert à chaud.

---

015 — Accentuer les prompts, et lier le cache à leur version

Date : 13 octobre 2026 · **État** : accepté

Contexte. Défaut constaté en imprimant la première fiche de révision : les sorties du modèle revenaient **en français sans accents**. « Ce cours presente les ateliers du module… ». La cause n'était pas la bibliothèque PDF — vérifié, reportlab rend les accents sans difficulté — mais le texte stocké en base.

Le modèle imitait le style des consignes, elles-mêmes écrites sans accents par une convention de code destinée à la console Windows. Cette convention n'avait aucune raison de s'appliquer à du texte envoyé au modèle.

Décision. Un fragment `LANGUE` partagé, **lui-même accentué**, ajouté aux quatre prompts, qui passent en v3.

Et une seconde décision, plus importante. La version du prompt entre dans la clé de cache des résumés. Sans elle, cette correction serait restée invisible : les résumés déjà produits continuaient à s'afficher sans accents. Toute amélioration future des consignes se propage désormais d'elle-même.

Ce que ça coûte. Un changement de prompt invalide le cache et relance les analyses — donc consomme du quota. C'est le prix d'une correction qui prend effet.

Leçon. Un défaut invisible à l'écran peut être criant à l'impression. Ce sont les livrables qui sortent de l'écran — ici une fiche imprimée — qui révèlent ce genre de chose.

---

016 — Refuser un cours trop long, et plafonner les appels au modèle

Date : 13 octobre 2026 · **État** : accepté

Contexte. Une revue du projet a trouvé deux trous que la documentation contredisait.

Le premier. `routers/courses.py` coupait le contenu à `MAX_COURSE_CHARS` sans rien dire. Mesuré : 476 000 caractères envoyés, 200 000 stockés, **276 000 perdus**, HTTP 201, aucun avertissement. C'est exactement le défaut que le découpage avait corrigé plus loin dans la chaîne — mais à l'import, là où l'utilisateur regarde.

Décision. Refuser en 400, avec le nombre de caractères en trop et la limite. Plutôt que tronquer avec un avertissement : l'étudiant sait quoi faire, au lieu de croire son cours complet. Un cours de plus de 200 000 caractères — soixante pages — gagne de toute façon à être découpé.

Le second. `RATE_LIMITED` figurait dans la taxonomie depuis le début sans qu'aucune route ne l'émette. Un double-clic suffisait à déclencher deux générations de quiz ; dix clics épuisaient le quota du jour, au pire

moment.

Décision. Une fenêtre glissante en mémoire, dix appels par minute et par utilisateur, posée uniquement sur les trois routes qui appellent le modèle. Consulter ses cours ou sa progression ne coûte rien et n'est pas plafonné.

Ce que ça ne fait pas. Ne protège pas d'un abus déterminé : un utilisateur peut créer plusieurs comptes. L'objectif est d'éviter l'accident. Et sur plusieurs instances, il faudrait un magasin partagé.

Leçon. Les deux défauts étaient des **écarts entre la documentation et le code** : une garantie affirmée mais contournable, et un code d'erreur qui ne pouvait pas survenir. Ce sont les plus dangereux, parce que personne ne les surveille — on croit le problème traité.

---

017 — Lire aussi les `.docx` et les `.pptx`, et les lire correctement

Date : 13 octobre 2026 · **État** : accepté

Contexte. StudyAI n'acceptait que le PDF. Or les supports de ce module sont distribués en **PowerPoint** pour les cours et en **Word** pour les ateliers : sur les neuf fichiers de l'énoncé, aucun n'est un PDF. L'étudiant devait donc convertir chaque fichier à la main avant de pouvoir s'en servir — un travail que la machine fait mieux, et qui dégradait au passage la mise en page.

Décision. Trois lecteurs derrière une seule entrée, `services/extraction.py`. PyMuPDF pour le PDF, python-docx et python-pptx pour le reste : deux bibliothèques **Python pures**, donc rien à installer sur le poste — le même critère qui avait fait écarter weasyprint au profit de reportlab (décision 012).

Le format est choisi sur l'extension, pas sur le type MIME

Le navigateur annonce le type que le système d'exploitation lui souffle, et Windows envoie régulièrement `application/octet-stream` pour un `.pptx` selon l'application qui a créé le fichier. **Mesuré** : deux des cinq supports d'essai sont arrivés en `application/octet-stream`. Un contrôle sur le type MIME les aurait refusés.

L'extension, elle, est ce que l'étudiant voit. Et elle n'est pas le dernier mot : le format réel est vérifié juste après, par le lecteur lui-même — python-docx refuse une archive qui n'est pas un paquet Word. *L'extension oriente, le lecteur tranche.* Le même critère est appliqué côté navigateur (`accept=".pdf,.docx,.pptx"`), de sorte que les deux filtres ne peuvent pas diverger.

Trois défauts d'extraction, trouvés en sondant les vrais fichiers

Aucun des trois n'aurait produit d'erreur. Tous les trois auraient dégradé silencieusement la qualité des résumés — la pire catégorie de défaut sur ce projet.

Défaut	Mesure sur les supports du module
`document.paragraphs` **ignore les tableaux**	271 caractères sur 1 602 dans `Atelier 3.docx`, soit **17 %** — et dans un atelier, le tableau est la grille de notation
`slide.shapes` suit l'ordre d'**empilement**, pas de lecture	Diapo 2 de `Chapitre5.pptx` : titre → **numéro de page** → image → corps. Les 54 diapos injectaient 54 chiffres orphelins
Espaces de **largeur nulle** (`\u200b`) au milieu des mots	Présents dans `Prompt Engineering.pptx`. Invisibles, mais ils brisent un mot pour toute comparaison de chaînes

Le troisième mérite un mot : il n'est cosmétique qu'en apparence. Un caractère invisible dans un mot fait échouer le contrôle d'**ancrage du quiz** sur une citation parfaitement fidèle, et produit des jetons fantômes dans le découpage en mots de **BM25**. C'est le même piège que l'ancrage strict du banc d'essai (EVALUATION.md section 5) : la mesure et le texte doivent parler la même langue.

Les notes de l'orateur sont incluses — et c'est une décision de fond

Sur un diaporama de cours, les diapositives sont télégraphiques et l'explication vit dans les notes. **Mesuré sur `séance1-Introduction.pptx` : 42 diapositives sur 50 portent des notes, qui représentent 31 % du texte du support.** Les écarter, c'était résumer le squelette d'un cours en ignorant le cours.

Vérifié par un vrai appel : à la question « dans quels domaines autres que le logiciel l'IA est-elle présente ? », le tuteur répond « santé, finance, industrie » **et cite la note comme source**. Les trois mots sont absents des diapositives et présents dans les notes — contrôlé par programme. Sans les notes, le tuteur aurait refusé cette question, à juste titre et pour une mauvaise raison.

Le coût, honnêtement. Certaines notes sont des mémos de production et non de la pédagogie : `Chapitre5.pptx` contient « Logo EUR-ACE » et « Taille 54 ». Elles entrent dans le cours. Nous ne les filtrons pas, parce qu'aucune règle ne distingue un mémo d'une explication sans risquer d'écarter du contenu réel — et parce que 31 % de contenu utile vaut mieux qu'une poignée de notes de mise en page.

Le plafond de taille passe de 8 à 25 Mo

Un diaporama est très défavorable sur le rapport octets/texte : `séance1-Introduction.pptx` pèse 4,6 Mo pour 29 000 caractères utiles, soit **160 octets par caractère** — le reste est en images. Ce support passait sous l'ancien plafond, mais sans marge. Le format et la taille sont vérifiés **avant** la lecture des octets, ce qui préserve la correction de la décision 016.

Leçon. Les trois défauts d'extraction ont été trouvés en **sondant les vrais fichiers avant d'écrire le lecteur**, pas en relisant la documentation des bibliothèques. Un extracteur naïf aurait fonctionné sans jamais lever d'erreur, en perdant un tiers du cours.

---

018 — Examen blanc multi-cours, et pourquoi il ne coûte rien

Date : 13 octobre 2026 · **État** : accepté

Contexte. La boucle de révision s'arrêtait à « un cours, un quiz ». Or avant un examen réel, un étudiant révise **plusieurs chapitres à la fois**, et veut savoir s'il tient dans le temps imparti. C'était la pièce manquante entre « générer des questions » et « se préparer ».

La propriété centrale : zéro appel au modèle

Les questions viennent des **quiz déjà générés**, qui sont en base. Composer un examen de vingt questions sur quatre cours ne déclenche donc aucun appel : l'étudiant peut en passer dix dans l'heure sans entamer son quota. Sur un projet contraint par un palier gratuit, c'est la différence entre une fonctionnalité utilisable et une démonstration unique.

La génération des questions manquantes existe, mais elle est **demandée explicitement** (`completer: true`), annoncée comme payante dans l'interface, et soumise au plafond de débit. Jamais déclenchée en silence.

Cette propriété est **vérifiée par un test**, pas seulement affirmée : `test_composer_un_examen_n_appelle_pas_le_modele` compte les appels à la doublure avant et après. Une affirmation de coût qui n'est pas testée est une affirmation qui dérivera.

Les questions sont figées dans la ligne, pas référencées

Un examen aurait pu stocker des couples (quiz, rang). Il stocke une copie des questions. **Duplication assumée** : un quiz peut être régénéré ou supprimé, et un examen passé la semaine dernière doit rester corrigible exactement tel qu'il a été passé. Un examen dont la correction change après coup ne vaut rien comme trace de révision. La normalisation cède devant la reproductibilité.

Pour la même raison, la table `exams` est rattachée à l'**étudiant** et non à un cours : supprimer un cours ne doit pas emporter l'examen qui portait aussi sur les trois autres.

Le temps est mesuré, pas imposé

Le serveur fait foi sur la durée écoulée — il recalcule `remainingS` à chaque lecture, donc trafiquer l'horloge du navigateur ne donne rien. Mais une copie rendue en retard est **corrigée quand même**, et simplement marquée. Un outil de révision n'a aucune raison de punir : l'étudiant veut savoir s'il a dépassé, pas perdre son travail.

En revanche, une copie déjà rendue **ne peut pas l'être deux fois**. Sans ce garde-fou, il suffirait de lire le corrigé puis de renvoyer le formulaire.

Le déficit de questions est redistribué, et signalé

Demander dix questions sur deux cours dont l'un n'en a que trois en rend **dix**, pas huit : le déficit est repris sur le cours qui a du stock, par tours successifs pour ne pas faire porter tout le rattrapage au premier cours venu. Et l'étudiant est averti que l'équilibre n'a pas été tenu, avec le détail par cours.

Un filtre mesuré, puis écarté

Deux quiz successifs sur le même cours reposent souvent la même question. Le dédoublonnage compare les énoncés normalisés — accents, ponctuation et casse retirés — ce qui attrape les reformulations de surface.

Il laisse passer les **doublons sémantiques**. Observé en conditions réelles : « Quel est le prix de base par nuit pour une chambre standard *dans le fichier booking_engine.py* ? » et « … *selon le cours* ? » se sont retrouvées dans le même examen.

Un filtre par similarité de Jaccard semblait la réponse évidente.

> **Correction du 13 octobre, en fin de journée.** La première rédaction de cette décision justifiait le refus par une mesure : un doublon réel à 0,63 contre deux questions légitimes à 0,75, donc des populations qui se chevauchent. **Ce contre-exemple à 0,75 était inventé** — écrit à la main pour éprouver l'idée, pas tiré des données. Justifier un refus par un chiffre fabriqué est précisément le défaut que ce projet reproche ailleurs. La mesure a été refaite sur les données réelles ; elle ne dit pas la même chose, et la conclusion a changé de raisons.

Ce que disent les données réelles

Sur les 18 questions en base, soit 88 paires issues d'un même cours :

Seuil de Jaccard	Paires supprimées	Faux positifs observés
0,55	5	**0**
0,65	3	**0**
0,70	1	**0**

Les cinq paires au-dessus de 0,55 sont **toutes** de vrais doublons, jusqu'à 0,88 pour « Quel message d'erreur s'affiche… » contre « Quel est le message d'erreur affiché… ». Aucune paire légitime n'atteint ce niveau dans les données observées. **Le filtre est donc mieux fondé que la première rédaction ne le prétendait.**

Une seconde piste a été mesurée puis écartée pour de bon : dédoublonner sur la **citation source identique**, en supposant que deux questions tirées du même passage testent le même fait. Résultat : **3 doublons pour 3 faux positifs**, une pièce de monnaie. Deux questions parfaitement distinctes naissent couramment du même passage — « quel langage pour les scénarios ? » et « quel outil de test comportemental ? ».

Pourquoi le filtre n'est quand même pas en place

Trois raisons, dont aucune n'est celle qui avait été écrite :

1. **L'échantillon est minuscule** : 18 questions, 2 cours, un seul domaine. Calibrer un seuil là-dessus est du surapprentissage. Rien ne garantit que la fenêtre propre 0,55–0,65 survive à un cours de droit ou de mathématiques.
2. **L'erreur du filtre serait invisible.** Un doublon qui passe, l'étudiant le voit et hausse les épaules. Une question légitime supprimée, personne ne sait qu'elle a existé. Le bénéfice se constate, le coût jamais : c'est la pire forme possible pour une heuristique, parce qu'elle aura toujours l'air de fonctionner.
3. **L'enjeu reste faible** : un doublon coûte une question gâchée sur vingt.

La fréquence, en revanche, avait été sous-estimée : **5 paires proches sur 18 questions**, soit près d'une question sur trois impliquée. Sur un examen de dix questions, il faut s'attendre à un ou deux doublons.

Ce qu'il faudrait faire à la place

Ne pas supprimer, mais **déprioriser** : à la composition, préférer la question qui ressemble le moins à celles déjà retenues, et prendre quand même une question proche si le stock ne suffit pas. La raison n° 2 disparaît alors entièrement — rien n'est perdu en silence, l'examen garde son compte exact, et un doublon n'apparaît que faute d'alternative. Le seuil cesse d'être une suppression irréversible pour devenir une préférence. Non implémenté à ce jour.

Leçon. La première version de cette décision concluait juste pour une mauvaise raison, et c'est plus dangereux qu'une erreur franche : le raisonnement avait l'air mesuré. **Un contre-exemple construit par celui qui veut conclure n'est pas une mesure.** Il faut aller le chercher dans les données, même — et surtout — quand on a déjà son avis.

---

019 — Recherche et tuteur sur toute la bibliothèque

Date : 13 octobre 2026 · **État** : accepté

Contexte. Un étudiant qui révise ne sait pas toujours dans quel chapitre une notion a été vue — c'est souvent la vraie question : « où ai-je vu l'exécution symbolique ? ». StudyAI obligeait à ouvrir chaque cours et à interroger son tuteur l'un après l'autre.

Deux actions, deux coûts, dits explicitement

Route	Coût	Pourquoi
`GET /api/search`	**0 appel**	purement lexical, en local
`POST /api/ask`	**1 appel**, quel que soit le nombre de cours	la sélection des passages est faite avant, gratuitement

Dix cours ne coûtent donc pas dix fois plus cher qu'un seul. Les deux coûts sont **affichés dans l'interface** : sur un palier gratuit, c'est la seule façon pour l'étudiant de savoir ce qu'il consomme.

Et la recherche fonctionne **sans clé d'API** — donc pendant une panne du fournisseur, ce qui est exactement le moment où l'on veut encore pouvoir réviser.

Le BM25 est extrait, pas recopié

La formule vivait dans `selectionner`, au service du tuteur d'un cours. Elle est désormais dans `scores_bm25`, partagée. Deux implémentations de la même formule auraient fini par diverger, et la seconde n'aurait été testée par personne — un test le vérifie explicitement.

Le déplacement a révélé un **coût quadratique** : la fréquence documentaire de chaque terme était recalculée pour chaque segment, soit O(segments^2 × termes). Invisible sur les trois segments d'un cours, sensible sur les segments de toute une bibliothèque. Elle est maintenant calculée une fois.

L'IDF est calculé sur toute la bibliothèque, pas cours par cours. C'est le point : un mot banal dans un cours mais rare dans les autres pèse alors correctement, ce qui est précisément le signal cherché quand on demande « dans lequel de mes cours ai-je vu ça ? ».

Le découpage est refait à chaque recherche

Les segments ne sont stockés nulle part. Les conserver demanderait une table de plus et une invalidation à chaque modification de cours, pour économiser une suite d'expressions régulières sur du texte déjà en mémoire. Le compromis est assumé ; il deviendrait discutable bien au-delà du volume qu'un étudiant accumule en un semestre.

La limite lexicale, mesurée puis réduite

La recherche compare des **mots**, pas du sens. Une requête qui n'emploie aucun mot du cours ne trouve rien, même si la notion y est (DECISIONS 005).

Un cas précis est apparu à l'essai : **« binome » ne trouvait rien dans** `Atelier 3.docx`, qui écrit pourtant « le travail se fait en binômes ». Pas de racinisation : les jetons sont comparés entiers, et `binomes ≠ binome`. C'est le ratage le plus fréquent en français, et le plus déroutant — l'étudiant voit le mot de ses propres yeux dans son document.

Corrigé côté requête uniquement : chaque terme cherché est élargi à ses variantes singulier/pluriel. Le choix de l'endroit est délibéré — `scores_bm25` est partagée avec la sélection des passages du tuteur d'un cours, dont les mesures sont publiées dans EVALUATION.md. Modifier la façon dont les termes s'apparient invaliderait ces chiffres ; élargir la requête ne change rien pour l'appelant qui ne le demande pas.

Ce n'est **pas** une racinisation : « tester » ne trouvera toujours pas « test ». La limite est réduite à sa forme la plus gênante, pas supprimée.

Les réponses transversales ne sont pas enregistrées

Un échange qui traverse plusieurs cours ne se rattache à aucun. L'accrocher au cours le plus représenté serait une approximation visible dans l'historique, et créer une neuvième table pour un échange que l'étudiant poursuivra de toute façon dans le tuteur du bon cours ne se justifie pas. Les sources nommant leur cours, le chemin de reprise existe.

Zéro passage : répondre sans appeler

Quand aucun passage ne correspond, la réponse « non couverte » est produite **sans appel au modèle**. Ce n'est pas qu'une économie : c'est plus juste. Le modèle n'aurait eu aucun passage à lire, aurait conclu la même chose, avec en prime le risque de compléter par ses connaissances générales.

Leçon. Les deux décisions les plus structurantes de ce module portent sur l'**endroit** d'un changement, pas sur le changement lui-même : extraire BM25 plutôt que le recopier, et élargir la requête plutôt que la fonction de score. Au mauvais endroit, les deux auraient marché — et abîmé autre chose.

---

020 — Tableau de consommation : montrer ce que l'IA coûte

Date : 13 octobre 2026 · **État** : accepté

Contexte. Toute la télémétrie était enregistrée à chaque opération depuis le premier jour — modèle, version de prompt, température, jetons, latence — et n'apparaissait **nulle part**. Des données collectées que personne

ne regarde ne servent à rien ; pire, elles donnent l'illusion d'une mesure.

Sur un module intitulé *IA for Software Engineering*, savoir ce qu'un appel coûte fait partie du sujet. Et sur un palier gratuit, c'est la seule façon de savoir où l'on en est avant de tomber sur un quota épuisé en pleine révision.

Une opération n'est pas un appel — et le compte était jeté

Le pipeline comptait déjà les appels réels (`Telemetrie.appels`) : un par segment, plus l'agrégation pour un résumé long. **Ce compte n'était pas persisté.** Et `segmentCount` ne permet pas de le reconstruire : pour un quiz, le nombre d'appels dépend aussi du nombre de questions demandé *et* de `SEGMENTS_QUIZ` , un paramètre de configuration qui a pu changer depuis. Une formule fragile aurait affiché un chiffre faux avec aplomb.

Migration 0004 : colonne `calls` , **nullable**. Les lignes antérieures restent à `NULL` et sont comptées **à part**, jamais fondues dans le total. Le tableau affiche « — » plutôt que « 0 ».

Trois partis pris contre la fausse précision

1. **Le tarif est affiché avec sa source.** Les prix viennent d'agrégateurs tiers consultés le 13 octobre, qui ne s'accordaient pas entre eux ; la page officielle de Google fait foi. Un chiffre d'argent présenté sans sa provenance se lit comme un fait.
2. **Les opérations sans décompte sont visibles**, pas lissées.
3. **Les économies du cache ne sont pas estimées.** Ce serait le chiffre le plus parlant, et rien ne l'enregistre. La section « ce qui n'a rien coûté » ne montre que ce qui est réellement compté : les examens blancs et les questions qu'ils ont servies.

Leçon. Les trois décisions de ce module portent sur ce qu'il ne faut **pas** afficher. Un tableau de coûts qui arrondit ses trous est pire que pas de tableau : il donne confiance.